

=====

JSON,STRUCTURED OUTPUT & PYDANTIC

=====

Ravi is 22 years old and he is learning Python

Name: Ravi

Age: 22

Course: Python

Unsrtructured data:

It does not follow a fixed field based structure

eg:

Ravi is 22 years old and he is learning Python

Ravi is learning Python and he is 22 years old.

Structured Data:

It is the information organized according to known structure.

Name: Ravi

Age: 22

Course: Python

Field+Value

The Samsung Galaxy S25 costs 75000 rupees and belongs to the smartphone category.

The Samsung Galaxy S25 costs INR 75000 and belongs to the smartphone category.

The Samsung Galaxy S25 costs Seventy Five Thousand Rupees and belongs to the smartphone category.

Name: Samsung Galaxy S25

Price: 75000

Category: smartphone

JSON: JavaScript Object Notation

It is very common structured-data format.

```
{  
  "name": "Ravi",  
  "age": 22,  
  "course": "Python"  
}
```

key:value

JSON can contain different data types

```
{  
  "name": "Durga",  
  "experience": 20,  
  "rating":4.8,  
  "active":true  
}
```

name--->String

experience -->Number

rating-->Number

active-->Boolean

JSON vs Python Dictionary:

JSON --->Data Interchange Format

Python Dict --->Python data type/Object

both looks similar but they are different concepts.

```
{  
  "Name": "Durga",  
  "rollno": 101,  
}
```

```
import os  
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
response = llm.invoke(  
    """  
    Give information about Python.
```

Return only JSON with these fields:

```
name
    creator
category
difficulty
"""
)

print(response.content)
```

E:\lcclasses>py test.py

```
```json
{
 "name": "Python",
 "creator": "Guido van Rossum",
 "category": "Programming Language",
 "difficulty": "Easy to Moderate"
}
```
```

"Return only JSON" may not enough because:

- a required field is missing

- field names may be change

- wrong type may be assigned

- our program expects one structure but gets another

We should require another concept: Schema

What is Schema:

A schema describes the structure of the data that our application expects.

database student table:

sid -->INTEGER

name-->VARCHAR

age -->INTEGER

course-->VARCHAR

Student:

name :str

age :int

course :str

schema contains Fields and Types

Fields-->name,age,course

Types-->str,int

Field types are very important. It makes sure our expected data structure clearer.

What is Pydantic:

Pydantic is a Python library that helps us define data models using type annotations and validate data against those models.

Pydantic ---> Define Structure(Schema) and Validate Data

Installation:

```
pip install pydantic
```

```
from pydantic import BaseModel
```

```
from pydantic import BaseModel
```

```
class Student(BaseModel):
```

```
    name:str
```

```
    age:int
```

```
    course:str
```

```
student = Student(  
    name="Ravi",  
    age=22,  
    course="Python"  
)
```

```
print(student)  
print(student.name)  
print(student.age)  
print(student.course)
```